

Rélatório de IA da busca no labirinto

Anderson Rodrigues dos Santos
Maíra Beatriz de Almeida Lacerda

31 de outubro de 2025

1 Introdução

O desafio de encontrar o caminho ideal em um labirinto é um problema clássico em Inteligência Artificial, especialmente no estudo de algoritmos de busca. Este relatório apresenta o progresso da implementação de algoritmos de busca informada e não informada aplicados à resolução de labirintos. O objetivo principal é utilizar essas abordagens para identificar o caminho entre o ponto de partida e o objetivo dentro de um ambiente delimitado por obstáculos.

Além da implementação, este trabalho contempla a realização das medições de desempenho exigidas, como tempo de execução, uso de memória, completude, optimalidade e número de nós gerados/expandidos. Essas métricas são fundamentais para avaliar a eficiência e a eficácia dos algoritmos utilizados, conforme especificado na descrição do trabalho.

2 Metodologia

2.1 Representação do Problema

O labirinto foi modelado como uma grade bidimensional (*grid*), onde cada célula representa um possível estado do agente. Cada estado é definido por um par (x, y) correspondente à posição no mapa.

- **Estado:** posição (x, y) do agente.
- **Ações:** mover-se para Norte, Sul, Leste e Oeste.
- **Transição:** movimento permitido apenas se a célula alvo não for uma parede e estiver dentro dos limites do mapa.
- **Custo:** custo unitário para cada movimento.
- **Teste de objetivo:** posição atual igual à posição do objetivo (G).

O mapa é definido a partir de um arquivo texto (`labirinto.txt`), com a seguinte convenção:

- S — posição inicial;
- G — objetivo;
- # — parede (obstáculo);
- . — célula livre.

2.2 Heurística

Para as buscas informadas, utilizamos duas heurísticas clássicas em grades 4-conectadas (N, S, L, O) com custo unitário:

Distância Manhattan

$$h_{\text{Manh}}(n) = |x_n - x_G| + |y_n - y_G|$$

É admissível e consistente nesse domínio, pois nunca superestima o custo real e obedece à desigualdade triangular.

Distância Euclidiana

$$h_{\text{Eucl}}(n) = \sqrt{(x_n - x_G)^2 + (y_n - y_G)^2}$$

Também é admissível e consistente com movimentos ortogonais de custo unitário. Note, porém, que no grid 4-direções vale $h_{\text{Eucl}}(n) \leq h_{\text{Manh}}(n)$. Logo, a Euclidiana tende a ser menos informada que a Manhattan (expande um pouco mais nós em A^*), embora ambas preservem a otimalidade do A^* .

2.3 Implementação da Busca em Largura (BFS)

A **Busca em Largura** (*Breadth-First Search* — *BFS*) foi implementada em C++ utilizando uma fila (*queue*) para a fronteira e um conjunto (*set*) para armazenar os estados explorados. O algoritmo expande os nós em ordem crescente de profundidade, garantindo completude e optimalidade em ambientes com custo unitário.

- **Critério de expansão:** ordem FIFO (primeiro a entrar, primeiro a sair).
- **Estruturas de dados:** `queue<Node*> frontier`, `set<State> explored`.
- **Métricas registradas:** nós gerados, nós expandidos, memória máxima e custo do caminho.

```
function BreadthFirstSearch(problem):
    node ← Node(problem.initial_state)
    frontier ← FIFO queue containing node
    explored ← {}

    while frontier not empty do:
        node ← frontier.pop()
        if goal_test(node.state) then return solution(node)
        add node.state to explored
        for each action in actions(node.state):
            child ← result of action
            if child not in frontier and not in explored then
                frontier.push(child)
    return failure
```

2.4 Implementação da Busca Gulosa (Greedy Best-First Search)

A **Busca Gulosa** foi implementada em C++ com o uso de uma `priority_queue`, onde os nós são ordenados pelo valor da heurística $h(n)$. O algoritmo prioriza os estados mais próximos do objetivo, segundo a estimativa da heurística, sem considerar o custo real do caminho percorrido.

- **Função de prioridade:** $h(n)$ (Distância Manhattan e Distância Euclidiana).
- **Estruturas de dados:** `priority_queue<pair<int, Node*> frontier`, `set<State> explored`.
- **Métricas registradas:** nós gerados, nós expandidos, memória máxima e custo total.

```
function GreedyBestFirstSearch(problem):
    frontier ← priority queue ordered by h(n)
    explored ← {}
    add start node to frontier

    while frontier not empty do:
        node ← frontier.pop()
        if goal_test(node.state) then return solution(node)
        add node.state to explored
        for each action in actions(node.state):
            child ← result of action
            if child not in explored:
                frontier.push(child)
    return failure
```

2.5 Implementação da Busca A* (A-Star)

A **Busca A*** foi implementada em C++ utilizando uma `priority_queue` como fronteira, ordenada pelo valor $f(n) = g(n) + h(n)$, onde $g(n)$ é o custo acumulado desde a origem e $h(n)$ é a estimativa heurística até o objetivo (utilizamos a distância Manhattan como heurística admissível e consistente). A A* é completa e ótima quando h é admissível, e tende a expandir menos nós que a Busca em Largura quando a heurística é bem informada.

- **Função de avaliação:** $f(n) = g(n) + h(n)$ (com h = Manhattan e h = Euclidiana).
- **Critério de expansão:** menor $f(n)$ na fronteira (desempate opcional por menor $h(n)$).
- **Estruturas de dados:** `priority_queue<pair<int, Node*> > frontier, unordered_map<State, int> g_cost, set<State> closed`.
- **Propriedades:** completa e ótima se h for admissível/consistente; custo computacional dependente da qualidade de h .
- **Métricas registradas:** nós gerados, nós expandidos, memória máxima (tamanho máximo da fronteira e do conjunto fechado), custo do caminho $g(goal)$ e profundidade da solução.

```
function AStarSearch(problem):
    start ← Node(problem.initial_state, g=0, h=h(initial))
    frontier ← priority queue ordered by f = g + h
    frontier.push( (start.g + start.h, start) )
    g_cost[start.state] ← 0
    closed ←

    while frontier not empty do:
        (_, node) ← frontier.pop()
        if goal_test(node.state) then return solution(node)

        if node.state in closed then continue
        add node.state to closed

        for each (action, step_cost, succ_state) in successors(node.state):
            tentative_g ← node.g + step_cost
            if succ_state not in g_cost or tentative_g < g_cost[succ_state] then
                g_cost[succ_state] ← tentative_g
                h ← heuristic(succ_state)
                child ← Node(succ_state, parent=node, g=tentative_g, h=h)
                frontier.push( (child.g + child.h, child) )

    return failure
```

2.6 Implementação da Busca em Profundidade (DFS)

A **Busca em Profundidade** (*Depth-First Search* — *DFS*) foi implementada em C++ utilizando uma pilha (stack) para a fronteira e um conjunto set) para registrar estados visitados e evitar ciclos. O algoritmo expande preferencialmente os nós mais profundos antes de retroceder (*backtracking*). Embora seja eficiente em termos de memória, é *não-ótimo* e pode ser *incompleto* em espaços infinitos ou com profundidade ilimitada.

- **Critério de expansão:** ordem LIFO (último a entrar, primeiro a sair).
- **Estruturas de dados:** `stack<Node*> frontier, set<State> visited`.
- **Propriedades:** baixo uso de memória ($O(b \cdot m)$), não garante optimalidade; pode exigir limite de profundidade para término garantido.
- **Métricas registradas:** nós gerados, nós expandidos, memória máxima e custo/profundidade da solução (quando encontrada).

```
function DepthFirstSearch(problem):
```

```

node ← Node(problem.initial_state)
frontier ← LIFO stack containing node
visited ←

while frontier not empty do:
    node ← frontier.pop()
    if goal_test(node.state) then return solution(node)

    if node.state in visited then continue
    add node.state to visited

    for each (action, step_cost, succ_state) in successors(node.state) do:
        if succ_state not in visited then
            child ← Node(succ_state, parent=node,
                        g=node.g + step_cost)
            frontier.push(child)

return failure

```

2.7 Métricas de Avaliação

As métricas coletadas para comparação foram:

- Tempo de execução (ms);
- Nós gerados;
- Nós expandidos;
- Memória máxima utilizada (fronteira + explorados);
- Completude e optimalidade.

3 Experimentos e Resultados

Os experimentos foram realizados sobre um labirinto de 5x5 células, conforme exemplo a seguir:

```

S...
.###.
..#..
.###.
....G

```

Tabela 1: Comparativo de Algoritmos de Busca

Algoritmo	Tempo (s)	Memória	Nós Ger.	Nós Exp.	Custo	Solução	Ótimo
BFS	0.000084	17	18	15	8	SIM	SIM
DFS	0.000033	10	11	8	8	SIM	NÃO
Gulosa-Manh	0.000024	11	11	9	8	SIM	NÃO
Gulosa-Eucl	0.000058	11	11	10	8	SIM	NÃO
A*-Manh	0.000037	11	11	9	8	SIM	SIM
A*-Eucl	0.000044	14	14	13	8	SIM	SIM

Análise Comparativa:

- **Mais rápido:** Busca Gulosa (Manhattan) com tempo de execução de 0,000024 s.
- **Menor uso de memória:** DFS, com apenas 10 nós armazenados simultaneamente.
- **Custo ótimo (menor caminho):** 8 — obtido por **BFS, DFS, Gulosa-Manh, Gulosa-Eucl, A*-Manh** e **A*-Eucl**.

4 Análise e Discussão

A Busca em Largura (BFS) demonstrou comportamento previsível, garantindo sempre uma solução ótima quando as ações possuem custo unitário, devido à sua expansão uniforme em camadas. Entretanto, o custo computacional se mostrou mais elevado em termos de número de nós expandidos e uso de memória, o que é coerente com a literatura, já que o BFS armazena todos os nós de cada nível antes de prosseguir para o próximo.

A Busca em Profundidade (DFS) apresentou o menor uso de memória entre os métodos testados, uma vez que mantém apenas o caminho corrente na pilha. Seu tempo de execução também foi reduzido em cenários específicos, pois explora caminhos completos rapidamente. Contudo, o DFS *não é completo* em espaços de busca infinitos ou com ciclos (na ausência de controle de estados repetidos) e geralmente *não é ótimo*, podendo encontrar caminhos mais longos que o necessário ou até não encontrar solução se seguir um ramo incorreto.

A Busca Gulosa, tanto com heurística de *distância Manhattan* quanto com *distância Euclidiana*, foi o algoritmo mais rápido em todos os testes, priorizando a expansão dos nós aparentemente mais próximos do objetivo. A heurística Manhattan tende a gerar caminhos ligeiramente mais diretos em grades ortogonais, enquanto a Euclidiana fornece estimativas mais suaves e contínuas. Apesar disso, ambas desconsideram o custo acumulado ($g(n)$), o que faz com que a Busca Gulosa não garanta soluções ótimas — embora seu desempenho em tempo e memória seja o mais eficiente.

O algoritmo A*, utilizando as mesmas heurísticas, combina o custo já percorrido ($g(n)$) com a estimativa ao objetivo ($h(n)$), resultando em uma função $f(n) = g(n) + h(n)$. A distância Manhattan mostrou-se *admissível* e *consistente* para o ambiente em grade com movimentos apenas nas quatro direções cardeais, garantindo *completude* e *optimalidade*. Já a heurística Euclidiana também é admissível e apresentou comportamento semelhante, com pequenas variações no número de nós expandidos, mas mantendo a solução ótima.

De modo geral, observou-se que:

- A **BFS** e o **A*** (com Manhattan ou Euclidiana) obtiveram o mesmo custo de solução, confirmando a optimalidade teórica.
- A **Busca Gulosa** foi a mais rápida, porém sacrificou a optimalidade.
- O uso da heurística **Euclidiana** não alterou significativamente o custo final, mas pode gerar expansões mais suaves e menor variabilidade em mapas mais abertos.

5 Conclusão

Os resultados experimentais evidenciaram as diferenças fundamentais entre algoritmos de busca não informada (BFS e DFS) e informada (Gulosa e A*), além do impacto direto da heurística escolhida.

- **BFS**: garante solução ótima (com custo unitário) e completude, mas exige grande uso de memória e expande muitos nós.
- **DFS**: é mais econômica em memória e pode ser rápida em determinados casos, mas não garante completude nem optimalidade sem variações adicionais (como profundidade limitada).
- **Busca Gulosa**: alcançou o menor tempo de execução com ambas as heurísticas. A Manhattan apresentou leve vantagem em mapas com trajetos ortogonais, enquanto a Euclidiana tendeu a explorar mais suavemente regiões abertas. Nenhuma das duas é ótima, pois o algoritmo ignora o custo acumulado.
- **A***: combinando custo acumulado e estimativa heurística, manteve-se completo e ótimo em ambos os casos. O A* com Manhattan foi ligeiramente mais eficiente em tempo e memória, mas o A* Euclidiano obteve desempenho comparável.

Portanto, a escolha do algoritmo e da heurística deve considerar o equilíbrio entre *tempo de execução*, *memória disponível* e *qualidade da solução*. Em ambientes de grade com movimentos ortogonais, a heurística Manhattan tende a ser a mais apropriada. Já a Euclidiana pode ser mais vantajosa em domínios contínuos ou quando há movimento diagonal. Assim, o algoritmo A* se destaca como a melhor

opção geral, unindo eficiência e garantias de completude e optimalidade, enquanto a Busca Gulosa é preferível quando o tempo de resposta é o fator mais crítico.

Créditos e Declaração de Autoria

Autor: Anderson Rodrigues

Algoritmo feito: Busca de Aprofundidade (BDS) e Busca Gulosa.

Uso de IA: ChatGPT (OpenAI) foi utilizado apenas para revisão textual e formatação em LaTeX.

Uso de IA: Copilot (GitHub), Algumas partes do código foram geradas automaticamente.

Recursos externos: notas de aula da disciplina e exemplos fornecidos pelo professor.

Autora: Maíra Beatriz de Almeida Lacerda

Algoritmos implementados: Busca A* (A-Star) e Busca em Profundidade (DFS).

Uso de IA: ChatGPT (OpenAI) foi utilizado exclusivamente para revisão textual, formatação em LaTeX e padronização da descrição dos algoritmos.

Uso de IA: GitHub Copilot foi utilizado pontualmente para sugerir trechos de código durante a implementação.

Recursos externos: notas de aula da disciplina, exemplos fornecidos pelo professor e materiais de apoio disponíveis na plataforma Moodle do curso.

Referências

Referências

- [1] ANDERSON SANTOS. *Inteligência Artificial — Busca e 8 Rainhas*. Repositório GitHub: github.com/anderrrsantos/Inteligencia_Artificial_Busca_e_8_Rainhas. Acesso em: out. 2025.
- [2] OLIVEIRA, Tiago Alves de. *Inteligência Artificial – Engenharia de Computação (EC_IA_001 a 005)*. Centro Federal de Educação Tecnológica de Minas Gerais – CEFET-MG, Campus Divinópolis, 2025. Material de aula: EC_IA_004_Busca e EC_IA_004_Busca-Parte 2 (*Estruturas e Estratégias de Busca*).
- [3] RUSSELL, S. J.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 4. ed. Pearson, 2022. Cap. 3–4: Algoritmos de busca informada e heurísticas.
- [4] KORF, R. E. A* Search: *A Survey of Recent Advances*. *AI Magazine*, v. 37, n. 1, p. 9-22, 2016.
- [5] PEARL, J. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.
- [6] MICROSOFT. *Documentação C++ (MSVC)*. Disponível em: <https://learn.microsoft.com/pt-br/cpp/cpp/?view=msvc-170>. Acesso em: out. 2025.
- [7] cplusplus.com. *C++ Reference – Standard Library, Containers and Algorithms*. Disponível em: <https://cplusplus.com/>. Acesso em: out. 2025.